Setup EMP development environment
&
reference project

Table of contents

Introduction & Requirements **4**
Introduction 4
Requirements 4
Board overview **5**
SD card slots 5
Power and UART 5
Step 1: Create, configure & build the Petalinux project **7**
Preparation 7
Create the Petalinux project 7
Configure the Petalinux Project 7
Build the Petalinux Project 8
Package the Petalinux Project 9
Create a bootable SD-card 9
Step 2: Build the epos root filesystem for EMP **10**
Introduction 10
Build the root filesystem 10
Adding PetaLinux kernel modules 10
Step 3: Boot the EMP/ Trenz baseboard **12**
Establish serial connection via Putty 12
Step 4: libuio - UserspaceIO helper library **13**
Introduction 13
Installation 13
Software tools for EMP - empTools **14**
Introduction 14
Overview: I2C related empTools for peripheral control & monitoring 14
Overview: AXI4-lite related empTools for emp-firmware control & monitoring 16
LpGbtSw **18**
emp-firmware - a quick setup guide **19**
Introduction 19
Creating the project 19
Configuring the firmware 19
Exporting the hardware design 21
Creating a Device Tree overlay to program the FPGA **22**
Introduction 22
Workflow 22
Loading the firmware 23

Introduction & Requirements

Introduction

This guide provides a step-by-step solution for setting up the EMP development environment. The EMP v1, EMP v2 and TEBF0808 baseboards are supported along with a TE0807-03-4BE21-A SoM module. The guide shows how to set up the initial Petalinux project using the provided “Board Support Package” (BSP), how to create the Alma 9-based root filesystem (epos) and gives a brief overview of the emp firmware and the available software toolkits.
It is mandatory to keep the Vivado & Petalinux versioning consistent and it is recommended to use the latest versioning supported by Central DCS (Vivado & Petalinux 2023.2). Inconsistencies in tool versions can lead to a variety of problems.

Important Notice:
There are version dependencies between Petalinux and Vivado! Assume that using a specific PetaLinux version requires the hardware design to be exported from the related version of Vivado Design Suite! For further Information check “Prerequisites” in the “PetaLinux Tools Documentation”.

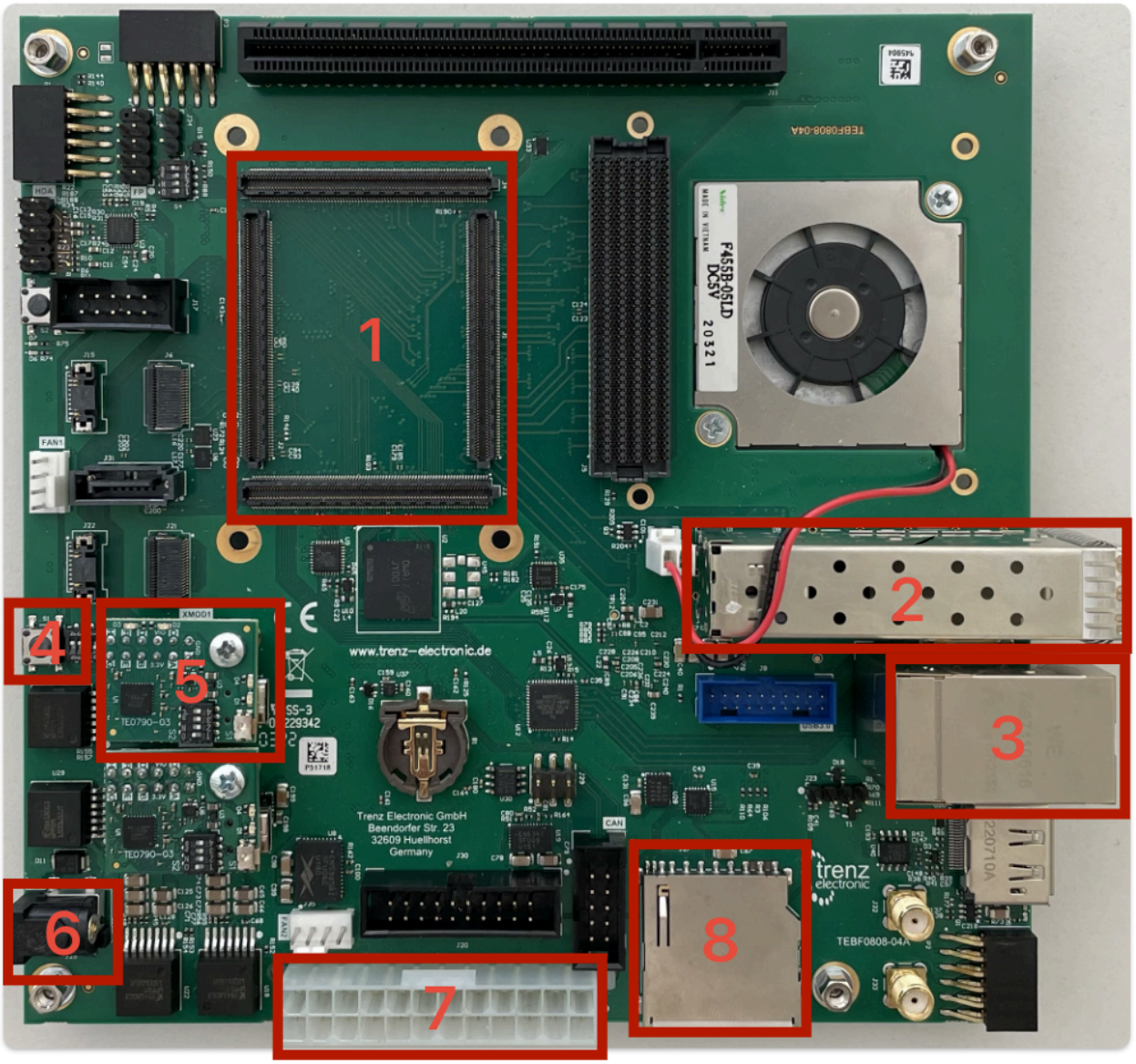
Please read the relevant documentation to install Petalinux and Vivado correctly.

Requirements

1) Hardware
    a) EMP v1, EMP v2 or Trenz TEBF0808 baseboard
    b) TE0807-03-4BE21-A SoM
    c) 10Gb/s SFP+ transceivers (IEEE 802.3-2012) or
    d) EMCI with **LpGBT v1**
    e) Break out style fiber
    f) SD-card (at least 16 GB)
2) Software
    a) Vivado 2023.2 (supported operating systems)
    b) Petalinux 2023.2 (installation requirements)

Board overview

Trenz Baseboard



Legend:

1. High speed connector to mount SoM
2. SFP+ cage
3. USB + Ethernet cage
4. Power button
5. JTAG & UART via micro-USB to USB-A
6. 12V Power Jack 2.1mm
7. ATX Power Connector (Main 12V & 5V supply)
8. SD-card slot

Note: There is a micro SD card slot on the back of the Trenz baseboard, which can be used instead of the SD card slot.

EMP Baseboard (v1 & v2)

Step 1: Create, configure & build the Petalinux project

Petalinux is a tool provided by AMD Xilinix for creating and deploying embedded Linux solutions on AMD processing systems. In this project, it is used to create bootables for the processing system of the MPSoC module, which contain the kernel configuration, kernel modules, U-Boot configuration and the Device Tree for the respective hardware platform.

Preparation

Assuming Petalinux 2023.2 is already installed on your machine, it is also necessary to clone the epos repository. The repository contains the necessary tooling to create the epos root filesystem as well as the Board Support Packages to build the Petalinux project. The Board Support Package can be understood as a template that defines how to support a particular hardware, in this case the EMP or Trenz baseboards.
Clone the repository into your workspace with your favorite verification method (ssh, https, krb5):

```
$ git clone ssh://git@gitlab.cern.ch:7999/emci-emp/epos.git
```

Create the Petalinux project

In order to be able to use the Petalinux tools, the necessary environment must first be sourced. Therefore move into the Petalinux installation location and run the respective shell script:

```
$ source <PetaLinuxDir>/settings.sh
```

Next create a directory inside your workspace where the project should be built & enter it. Now create the Petalinux project by running following command:

```
$ petalinux-create -t project -s <PathToBSP> -n <MyBootProj>
```

The placeholder *<PathToBSP>* describes the absolute path to the Board Support Package within the epos repository. Depending on the hardware for which the Petalinux project is created, the corresponding BSP for the EMP baseboard (*epos-bsp/emp-xV23-03-00-00/epos_emp.bsp*) or the Trenz baseboard (*epos-bsp/trenz-xV23-03-00-00/epos_trenz.bsp*) must be used. The placeholder *<MyBootProj>* symbolizes the name of the created project. After executing the command, a directory with the name *<MyBootProj>* is created and contains the project. This is then the workspace for all configuration and build processes. Now everything is ready to configure and then build the project.

Configure the Petalinux Project

Now enter the created *<MyBootProj>* directory, the workspace of the previously built Petalinux project. Next the command “petalinux-config –get-hw-description” is used, which allows to initialize or update a Petalinux project with hardware specific information from a Vivado hardware project via an exported Hardware Design file (.xsa). A hardware design is already available in the epos repository. Depending on the board used, select the EMP version (hw-description/emp-xV23-01-00-00/hw_desc.xsa) or the Trenz version (hw-description/trenz-xV23-03-00-00/hw_desc.xsa). The command only requires the specification of the directory of the hardware design, not the file itself! The command should therefore look like this:

```
$ petalinux-config --get-hw-description=../epos/hw-description/trenz-xV23-03-00-00/
```

After executing the command, the tool detects the changes in the imported hardware design and launches the Petalinux System Configuration Menu (Figure x).

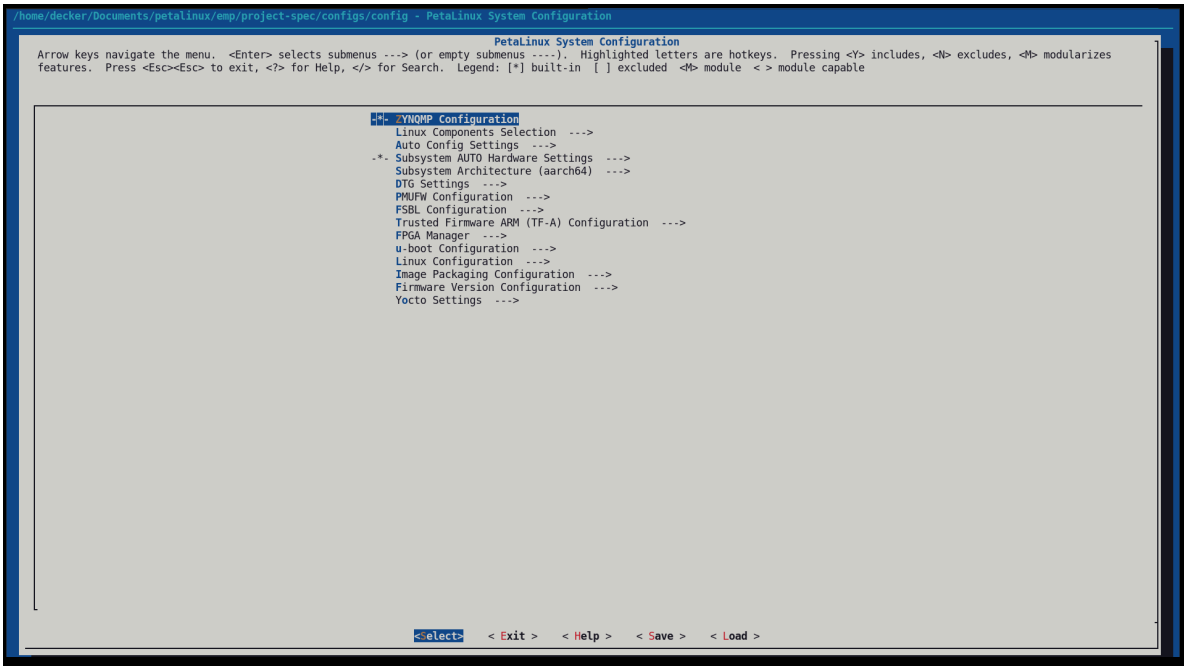


Figure: Petalinux System Configuration Menu

As long as the root file system is later deployed on the second partition of the SD card, no further configuration is required, since the necessary configuration is already preset through the BSP (“Root Filesystem Location” is set to *mmcblk1p2*). Further configurations such as the MAC address are also not necessary, as the MPSoC module has a unique address stored in a local eMMC, which is read out during booting and integrated into the device tree. The menu can therefore be closed directly and the project can be built.

Exception:

If it is intended to place the root filesystem on a “Network File System” (NFS) it is necessary to modify the "Kernel Boot Arguments". To edit them navigate to **DTG Settings -> Kernel bootargs**, ensure that “generate boot args automatically” is not selected and enter following string:

```
earlycon console=ttyPS0,115200 clk_ignore_unused root=/dev/nfs nfsroot=<NFS Server IP address>:<Location of NFS root directory>,tcp,nfsvers=3 ip=dhcp rw uio_pdrv_genirq.of_id=generic-uio
```

Here, the placeholders "*NFS Server IP address*" and "*Location of NFS root directory*" must be replaced with the information of the target server and the location of the root filesystem.

Save your configuration

After completing the configuration, you can exit petalinux-config by selecting "EXIT" at the top level menu. Before you finally exit the program, you will be asked whether the changes you have made should be saved or not. Select "Save" to retain the settings you have made.

Build the Petalinux Project

Now that the configuration is complete, the project can be built. To do this, execute the following command in the project workspace:

```
$ petalinux-build
```

The build process takes about 10 minutes on the first run. Afterwards, the build process will run faster, as

Petalinux does not create the project from scratch, but only applies changes in the configuration to the existing project.

Package the Petalinux Project

Once the project has been successfully built, the boot image can be packaged. If the project was created for a Trenz baseboard, no further steps are required and it can be packaged directly.

Exception:
If the project was created for an EMP baseboard, the Platform Management Unit (PMU) firmware built by Petalinux must be replaced by a custom-defined PMU firmware from the epos repository. Therefore, replace the pmufw.elf file in images/linux/ of the Petalinux workspace with the pmufw.elf file from the epos repository (epos/pmufw/emp-xV23-03-00-00/pmufw.elf).

To package the boot image run following command:

```
$ petalinux-package --boot --fsbl --fpga --u-boot --force
```

Create a bootable SD-card

Now that the Petalinux project has been successfully built and packed, the SD card must be prepared so that the boot files can be deployed on it. Depending on whether the root file system is also to be deployed on the SD card, two partitions must be created. On the other hand, if the root file system is located on an NFS server, only one partition on the SD card is required for the boot files.
Therefore, depending on the application, please partition as follows:

- root filesystem deployed on SD card:
    - Partition 1: minimum 1 GB FAT32 (to deploy the boot files)
    - Partition 2: minimum 8 GB EXT4 (to deploy the root filesystem)
- root filesystem deployed on an NFS server:
    - Partition 1: minimum 1 GB FAT32 (to deploy the boot files)

Help: Several tools can be used to partition a SD card (fdisk, gparted)

Then copy the following files from images/linux from your Petalinux workspace to the FAT32 partition of the SD card (boot partition):

- image.ub
- boot.src
- BOOT.BIN

Step 1 is now complete and the next step is to create the root filesystem!

Step 2: Build the epos root filesystem for EMP

Introduction

In Linux, directories are structured in a tree-like hierarchy. This directory structure provides easy isolation and identification of data. The root filesystem is at the top of the hierarchical file tree (known as '/'). It has a number of directories containing files critical for booting the system and system operations.
In the frame of the EMP Operating System (epos) the bootables are built apart from the root filesystem using Petalinux (Step 1). The corresponding root filesystem is built based on the latest AlmaLinux9 release from CERN IT. This part of the guide explains how to build the root file system using the tools available in the epos repository.

Build the root filesystem

Navigate to the epos-rootfs directory within the cloned epos repository. This will be the workspace to build the epos root filesystem.
First, binfmit_misc needs to be informed to use static QEMU. Therefore open the following configuration file (qemu-aarch64.conf) with your favorite text editor:

```
$ sudo nano -w /etc/binfmt.d/qemu-aarch64.conf
```

Add the following line to the file:

```
:qemu-aarch64:M::\x7fELF\x02\x01\x01\x00\x00\x00\x00\x00\x00\x00\x00\x00\x02\x00\xb
7\x00:\xff\xff\xff\xff\xff\xff\xff\x00\xff\xff\xff\xff\xff\xff\xff\xff\xfe\xff\xff\
xff:/usr/local/bin/qemu-aarch64-static:
```

To ensure that the changes are accepted and take effect, binfmt_misc must be restarted. To accomplish this, use the following command:

```
$ sudo systemctl restart systemd-binfmt.service
```

Finally, the epos file system can be created. The python program mkrootfs.py is used for this. It is executed using the command:

```
$ sudo python3 mkrootfs.py --root=<PATH> --extra=epos_rpms.txt
```

The argument --root with the placeholder <PATH> is used to specify the path where the root filesystem is to be deployed. Therefore, enter either the path to the ext4 partition of your SD card (rootfs partition) or the path from where you want to export your NFS file systems. The second argument --extra uses a text file that defines which rpms are additionally installed during the build process. An example of this can be found in the file epos_rpms.txt. It is recommended to use this option, e.g. to activate Quasar support on the EMP. Of course it is possible to add your own tools to the list which are useful for your purpose (e.g. nano, ifconfig,...).

Adding PetaLinux kernel modules

After the creation process has been successfully completed, it is necessary to import the Petalinux kernel modules into the newly created root file system. To do this, switch to the workspace of the Petalinux project from step 1.

```
$ cd <MyBootProj>/images/linux/
```

The rootfs.tar.gz data archive is located in the images/linux directory. Unpack it and then copy the folder 6.1.30-xilinx-v2023.2/, which is located in lib/modules/ of the unpacked data archive, into the directory lib/modules/ of the root filesystem.

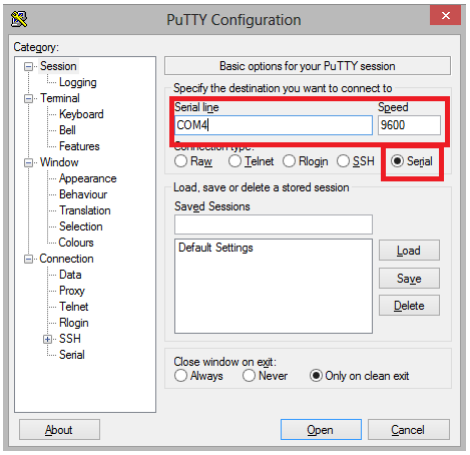
```
$ tar -xvf rootfs.tar.gz
$ cd lib/modules/
$ sudo cp -R 6.1.30-xilinx-v2023.2 /<location-of-epos-fs>/lib/modules/
```

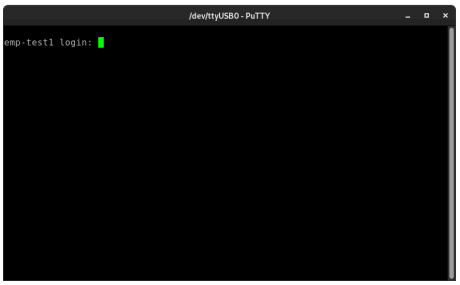
Step 2 is now complete and the EMP is ready to be booted!

Step 3: Boot the EMP/ Trenz baseboard

Now that the Petalinux project and the root file system have been built, the SD card can be inserted into the corresponding slot and the baseboard can be switched on. Since the MAC address is read from the eMMC during boot, it is currently not possible to access it easily. This means that if you are interested in working on the baseboard via an ssh connection and it is necessary to register the device in a database to access the network (e.g. network.db), it is mandatory to connect via the serial UART connection at the beginning.

Establish serial connection via Putty





Putty is a free and open-source terminal emulator, serial console and network file transfer application. Apart from supporting several network protocols (SCP, SSH, Telnet, rlogin) it can also be used to connect to a serial port.
When connecting to an EMP baseboard, check the kernel messages to see where the "USB Serial Device converter" has been connected (usually /dev/ttyUSB0). When connecting to a Trenz baseboard, two different "USB Serial Device Converters" are recognized and connected. The serial JTAG connection (normally /dev/ttyUSB0) is always connected to the lower port number and the serial UART connection (normally /dev/ttyUSB1) to the higher port number.
Now open putty with root rights, select "Serial" as the "Connection type", enter the port address (e.g. /dev/ttyUSB0) in the "Serial line" field and 115200 as the "Speed". Then click on "Open" and a terminal window will open.

Install Putty
The Unix source archive as well as the MSI Windows installer can be downloaded here: Putty download.

Step 4: libuio - UserspaceIO helper library

Introduction

The Userspace I/O System (UIO) was developed to reduce the effort required to develop kernel modules for all types of devices. Therefore, only interrupt handling and memory access are handled in the kernel, while the main part of the driver runs in userspace. This simplifies development and reduces the risk of serious errors in a kernel module. Please note that UIO is not a universal driver and is not suitable for devices that are already handled by other kernel subsystems (e.g. USB). UIO devices fulfill all of the following attributes:

- The device has a memory that can be mapped and can be controlled from it.
- The device uses interrupts
- The device does not fit into a standard kernel subsystems

Each UIO device is accessed via a device file and several sysfs attribute files. The device file is named /dev/uio0 for the first device, /dev/uio1 for the second and so on. The device file is used to access the address space of the device. Interrupts are handled by reading from the device file, a blocking read() returns when an interrupt occurs.
Within epos, uio is used to access registers of the AXI4 interface of an emp-lpgbt firmware block.

Installation

Libuio is a lightweight C programming library for detecting and binding UIO devices. The epos-kernel already has kernel-uio support enabled, the library itself is available in different versions (e.g. Digilent, H. Koch, ...). The preferred version is the [missinglinkelectronics]() by Benedikt Spranger, which is available on [gitlab]() including a cmake file for an easy build.
Clone the repository into your workspace with your favorite verification method (ssh, https, krb5):
Note: !!! Currently please also switch the branch from master into bugFix !!!

```
$ git clone ssh://git@gitlab.cern.ch:7999/atlas-dcs-emp/libuio.git
```

Next generate the build files from the CMake configuration file. Therefore enter the repository and execute:

```
$ cmake . -DCMAKE_INSTALL_PREFIX=/usr
```

Afterwards build the project and then install it:

```
$ sudo make install
```

Software tools for EMP - empTools

Introduction

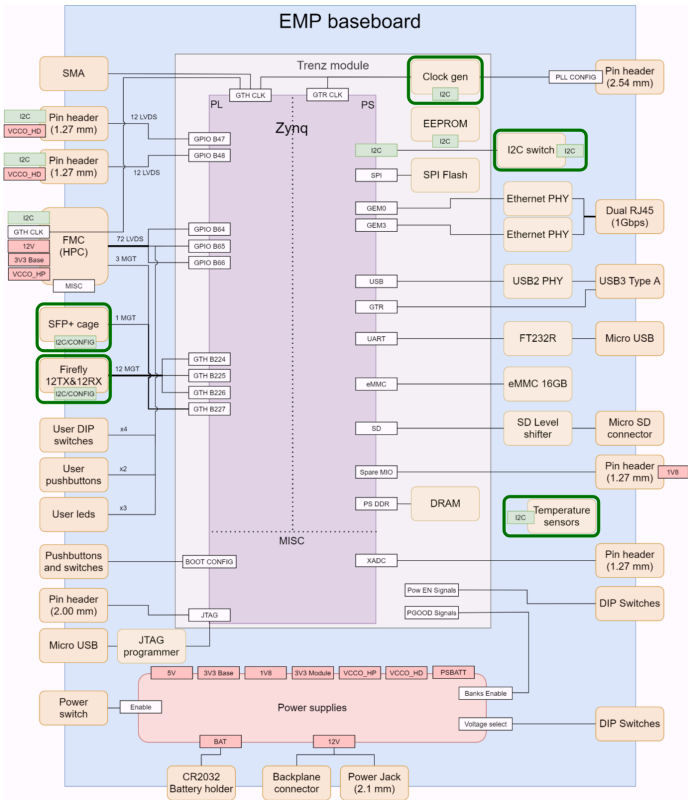
The empTools are a set of software tools for controlling, configuring and monitoring peripheral devices on the EMP baseboard as well as the emp-firmware. Not all software tools are also available for the Trenz baseboard as the peripheral devices differ between both in some respects. Therefore, only the necessary tools for the Trenz are available.
All sources for the empTools are available on gitlab as well as the artifacts from the CI/CD integration. The artifacts allow the users easy access to the binaries of the tools without a need of compiling them.

Requirements:
To execute the binaries some libraries need to be installed to the device:

- libuio
- boost-program-options

Overview: I2C related empTools for peripheral control & monitoring

I2C devices on emp baseboard



The emp baseboard uses an I2C master interface of the SoM PS to connect to a range of peripheral devices via an I2C extender. The I2C-related empTools allow the user limited access to the functions of these devices to read and write the internal registers and thus access measurements, alarms and warnings, but also to configure them. Limited means that only the necessary and useful functions are made available to the user and that the tools already provide predefined settings as part of the configuration of the devices, which can be used directly to program the I2C slave. The following devices are supported by empTools:

- FireFly TX and RX module
- Temperature Sensors
- SI5345 clock multiplier chip
- SFP+

clkProgrammer

The clkProgrammer tool can be used to program and monitor the integrated programmable clock multiplier chip SI5345. The configuration for this chip was created using the ClockBuilder Pro software from Skyworks. The software allows the user to go through all available settings of the chip and then export a C header. The header contains all the registers that need to be written via I2C to apply the configuration to the chip.

The tool accesses a configuration in order to grant the programmable logic the 320 MHz reference clock for executing the firmware. It is also possible to read out the chip's fault monitoring. This allows a number of values to be read that provide information about the operation of the chip and can be an indicator of unexpected behavior. The following monitoring information is provided:

- Loss of Signal (LOS) Status Monitoring
    - **LOS** status monitor for IN3 (bit 3), IN2 (bit 2), IN1 (bit 1) and IN0 (bit 0), indicating if a valid clock is detected. If the bit is set the input is lost.
    - **LOSXAXB** status monitor for the crystal or reference clock connected to the XA/XB pins.
    - **LOS FLG** status monitor sticky bits for IN3, IN2, IN1 and IN0. Sticky bits will remain asserted when a LOS event occurs until cleared. Writing zero to the bit will clear it.
    - **LOSXAXB FLG** status monitor sticky bits for XA/XB. Sticky bits will remain asserted when a LOS event occurs until cleared. Writing zero to the bit will clear it.
- Out-of-Frequency (OOF) Status Monitoring
    - **OOF** status monitor for IN3, IN2, IN1, IN0. Indicates if a valid clock is detected or if a OOF condition is detected.
    - **OOF FLG** status monitor sticky bits for IN3, IN2, IN1, IN0. Stick bits will remain asserted when an OOF event occurs until cleared. Writing zero to the bit will clear it.
- Loss of Lock (LOL) Status Monitoring
    - **LOL** status bit that indicates if the DSPLL is locked to an input clock
    - **LOL FLG** sticky bits for LOL register. Writing 0 to a sticky bit will clear it.

Following a short overview on the program options available to the tool:

-h [--help]	Shows the help for the clkProgrammer tool
-c [--configure]	This option is used to write the configuration register of the SI5345-chip to apply the configuration.
-i [--read_mon_info]	This option is used to display the monitoring informations of the SI5345-chip

tempSensor

There are 3 temperature sensors on the emp baseboard near critical areas of the board. They are located below the SoM, between the FireFly modules and near the DC/DC converter. The IC is a low-power temperature switch with an I2C interface and configurable resolution between 8-12 bits. Therefore, the device must be configured before the temperature can be read out. The tool provides a predefined configuration that is written to the device. The following options are available:

-h [--help]	Shows the help for the fireflyI2C tool
-d [--device] arg	Select the sensor (0 = Sensor under SoM, 1 = Sensor between FireFly, 2 = Sensor close to DC/DC)
-c [--configure]	Configure the selected device (needs a device selected via the -d option)
-r [--read_temp] arg	Read measured temperature (needs a device selected via the -d option)

fireflyI2C

The fireflyI2C tool can be used to monitor, control and reset the TX and RX FireFly module. The tool ensures that the Two Wire Interface (TWI) for both modules is enabled to allow I2C communication with the devices. Following a short overview on the program options available to the tool:

-h [--help]	Shows the help for the fireflyI2C tool
-m [--module] arg	This option is used together with an argument to select which module (TX or RX) is to be addressed. To select the TX module, enter 0 and to select the RX module, enter 1 as an argument.
-i [--read_ddm]	This option is used to read the digital monitoring information of one of the FireFly modules. It must be used together with the -m option to specify the module to be read.
-c [--channel] arg	The -c option is used to specify a channel and must always be used to deactivate,activate or to set its amplitude.
-d [--disable]	The -d option is used to disable the TX and RX channel of a emp-lpgbt IP communication line. If a channel is already disabled the command will have no effect.
-e [--enable]	The -e option is used to enable the TX and RX channel of a emp-lpgbt IP communication line. If a channel is already enabled the command will have no effect.
-a [--amplitude] arg	The -a option is used to set the level of the RX signals output amplitude. It accepts the arguments ‘low’, ‘med’ and ‘high’.
-r [--reset]	This option resets both the RX and the TX module. Reset means that all settings are initialized to the standard.
--read_temp	This option directly returns the temperature of a module, which needs to be specified with the -m option.

sfpI2C

The sfpI2C tool provides functionality to read the device Digital Diagnostic Monitoring register page, providing information on temperature, device supply, tx bias, tx power, rx power, status, alarms and warnings. Furthermore, the transmitter can be enabled or disabled. Following options are available:

-h [--help]	Shows the help for the fireflyI2C tool
-d [--disable_tx]	
-e [--enable_tx]	
-i [--read_ddm]	

Overview: AXI4-lite related empTools for emp-firmware control & monitoring

The empTools also provide two tools that act as an interface to the emp firmware via an AXI4-lite register map to obtain monitoring data about the status of an emp-emci pipeline and to send reset signals to a dedicated emp-emci pipeline. The tools were created using the PuzzledLizardWizard, a software that assists the developer in creating a lower level HAL in C++ including libuio integration. More information on how to use this tool is available on the related Gitlab page.

fwReset

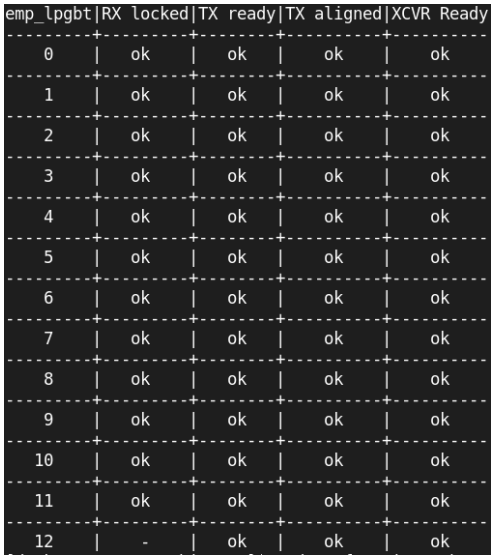
The fwReset tool can be used to interface the empfw_debughub IP inside the emp-firmware in order to send a reset signal. This means that the user can send a reset signal to a dedicated emp-emci pipeline to reset the emp_lpgbt IP (including the lpgbt_fpga IP and the GBTx IP) as well as the TX and RX unit of a transmission channel.
When using the emp_lpgbt option, the emp_lpgbt fabric is reset first, followed by the transmitter and then the receiver. If the tx_path option is used, only the sender is reset, while the rx_path option resets the

receiver. The --id option is to be used alongside the --ip option to specify the pipeline or fabric which should be reseted.

-h [--help]	Shows the help for the fireflyI2C tool
--ip arg	Specifies the type of IP block to be reset [‘emp_lpgbt’, ‘tx_path’, ‘rx_path’]
--id arg	Specifies the exact pipeline of an IP block that is to be reset [0-12]

fwMonitor

The fwMonitor is a tool that allows the user to check the status of each emp-emci pipeline. If the program is called without options and arguments, an overview of all pipelines is displayed. An example of this can be seen in the figure. The --id [arg] option can be used to display only the information of a single channel.



emp_lpgbt	RX locked	TX ready	TX aligned	XCVR Ready
0	ok	ok	ok	ok
1	ok	ok	ok	ok
2	ok	ok	ok	ok
3	ok	ok	ok	ok
4	ok	ok	ok	ok
5	ok	ok	ok	ok
6	ok	ok	ok	ok
7	ok	ok	ok	ok
8	ok	ok	ok	ok
9	ok	ok	ok	ok
10	ok	ok	ok	ok
11	ok	ok	ok	ok
12	-	ok	ok	ok

The overview contains 4 attributes for each pipeline:

- **“RX locked”** describes if the header of the received lpgbt frame is locked.
- **“TX ready”** describes whether the transmission logic of the downlink instance within the emp_lpgbt IP is in a reset state or not.
- **“TX aligned”** describes whether the alignment of the TXREFCLK in the transmitter has been successfully finished (detailed Information: Tx Phase Aligner for Xilinix transceiver).
- **“XCVR ready”** describes whether either the transmitter or receiver is in the reset state or whether they are ready for operation.

-h [--help]	Shows the help for the fwMonitor tool
-id arg	Specifies the monitor information of which channel to be displayed

Systemd-services

The emp-tools repository does also contain two linux systemd-service. Systemd services are used to start and stop services. The systemd command systemctl is used for control.

LpGbtSw

Introduction

The LpGbtSw library is a C++ based library aimed to provide software support to the lpGBT chip which is agnostic to the actual communication medium. The functional scope of the LpGbtSw is wide:

- a generic polymorphic API for register-based access to the lpGBT, independently from the communication medium (so called register clerk interface)
- an implementation of the register clerk interface for the actual EMP lpGBT communication channel, i.e. using the GBT-SC IC adapter(see section 6.3.2 and 6.3.3). This is achieved using the libuio library.
- a lpGBT simulator which is especially helpful for test and development of the LpGbtSw without having physical access to EMP and/or EMCI.
- an API to talk to the lpGBT slow-control peripheral interfaces - e.g. ADC, GPIO - encapsulating their register map intricacies
- a set of tools called ”demonstrators” which permit to use different LpGbtSw functions with stand-alone programs

The primary application of the LpGbtSw is to serve as Hardware Abstraction Library for the LpGbt OPC UA server.

LpGbtSw Demonstrator tools

As already mentioned, the LpGbtSw contains a number of so-called demonstrator tools. These tools can be used to

- read/write the configuration of the lpGBT
- read out the internal ADC of the lpGBT
- read/write the GPIOs controlled by the lpGBT on the EMCI
- read or set individual registers within the lpGBT

The binaries are available to users as CI/CD artifacts like the empTools and can be downloaded directly.

emp-firmware - a quick setup guide

Introduction

The emp firmware is a firmware framework offered by Central DCS. This reference project provides a set of IP blocks covering the communication path from PS (via registers) to the MGT lanes (connected to the FireFly modules). For more information, please refer to the slides shown during the Final Design Review. The following guide will take you through all the steps from cloning the Git repository, configuring the board related settings, compiling the firmware and finally exporting the required hardware design (.xsa) to allow the generation of a Device Tree Overlay (see Section xx).

Creating the project

Clone the repository into your workspace with your favorite verification method (ssh, https, krb5):

```
$ git clone ssh://git@gitlab.cern.ch:7999/emci-emp/emp-firmware.git
```

Next navigate to the emp-firmware directory, source the Vivado settings and afterwards open Vivado:

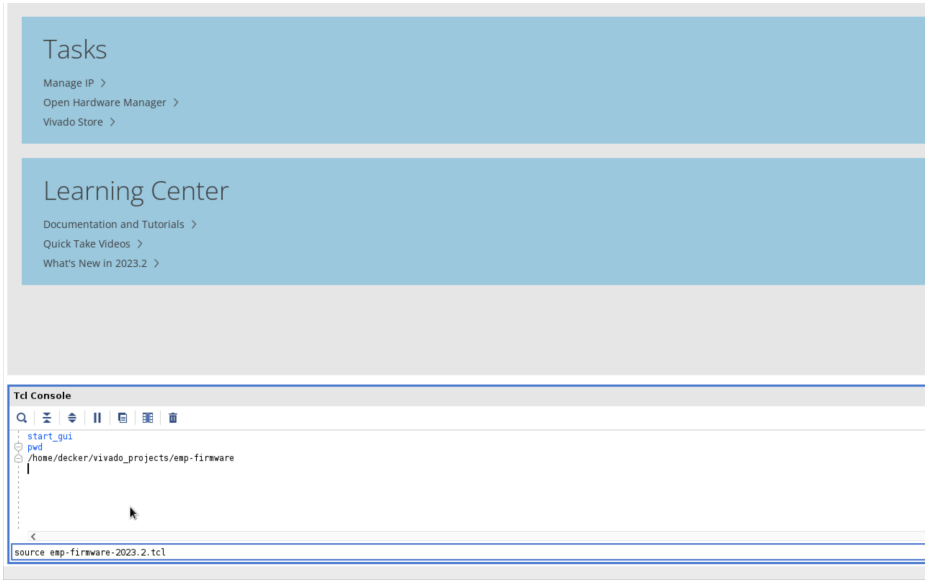
```
$ cd emp-firmware
```

```
$ source /<Vivado Installation Dir>/settings64.sh
```

```
$ vivado
```

Now call up the Tcl console, use "**pwd**" to ensure that is located in the emp-firmware directory and execute the following command (Figure xx):

```
source emp-firmware-2023.2.tcl
```



This rebuilds the project and then opens it. From now on, it is no longer necessary to execute the tcl command again, as the project has already been built and the tcl script would therefore fail. From now on, the project should appear in the project list on the right-hand side of the Vivado welcome window and can be opened from there.

Configuring the firmware

After the project has been created and opened, it may be necessary to change the configuration of the transceiver ip, depending on which board is used. Between v1 and v2 of the EMP baseboard, the routing

of the differential lines between SoM and FireFly modules has been optimized, which requires a corresponding configuration of the polarity settings.
Therefore, open the block design and click on the transceiver_firefly_v1 block to reconfigure the IP. A window opens in which the user can set the polarity for all 12 channels using a hex string.
Please enter for the Trenz baseboard and the EMP baseboard v1:

- MGT RX Polarity Inv: 0xFFF
- MGT TX Polarity Inv: 0x000

For the EMP baseboard v2, please enter :

- MGT RX Polarity Inv: 0xE07
- MGT TX Polarity Inv: 0xFFF

Then click OK and next run “Generate Bitstream”.
Note: A critical warning is displayed during the implementation of the firmware. This warning can be ignored as it refers to an unconnected input port on the emp_lpgbt IP that can be used by users to connect their project-specific logic.

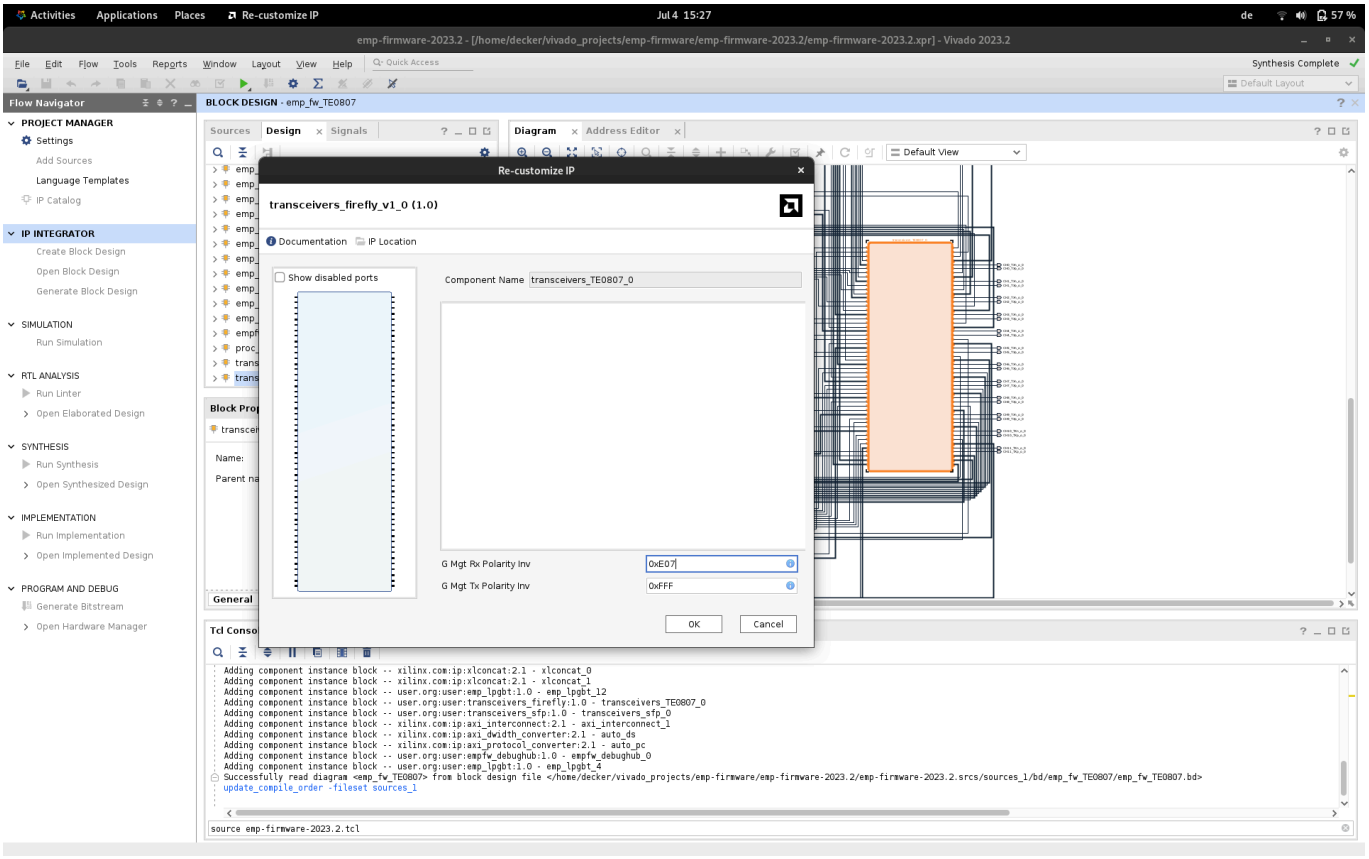
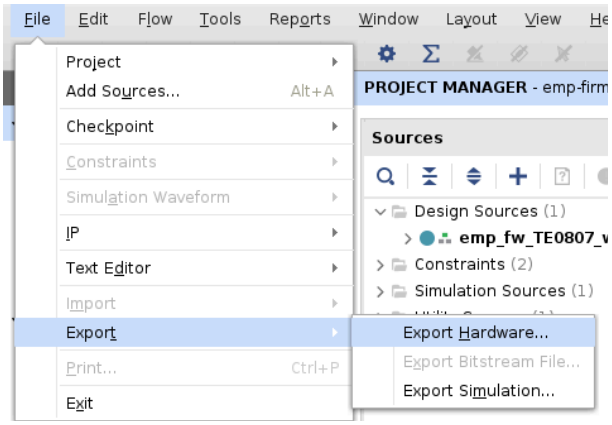


Figure xxx

Exporting the hardware design

Once the firmware has been successfully implemented, the last step is to export the hardware design including bitstream to an .xsa file. Navigate to File -> Export -> Export Hardware. The "Export Hardware Platform" window then opens. Click on “Next” and select "Include Bitstream". Then click on “Next” again, enter a file name and select the storage location to which the file is to be exported. Then click on “Next” again and finally on “Finish”.
Now the hardware design will be exported and the device tree overlay can be created.



Creating a Device Tree overlay to program the FPGA

Introduction

The Device Tree Overlay (DTO) is used to reprogram an FPGA while Linux is running. The DTO will add the child node and the fragments from the .dtbo file to the system's base device tree. The added device node will be probed after bitstream programming.
The Device Tree Overlay contains:

- The FPGA region to be programed
- The firmware file name
- The child nodes which correspond to the hardware being loaded into the FPGA region

Workflow

To simplify the workflow when creating a Device Tree Overlay for EMP, there are so-called etools in the epos repository. The assist the user while creating the Device Tree Overlay as well as loading the firmware later on EMP itself.
Therefore to create the Device Tree Overlay navigate to /etools/xsa-to-overlays/xsa-to-overlays and place the .xsa file, which was exported from the emp-firmware project, there. The python program emp-xsa-to-overlays.py is allowing the user to easily create the Device Tree Overlay. To be executed the program needs some tools, which are by default, not set as environment variables. Therefore it is necessary add following path to the environment variable in order to use xsct:

```
export PATH=$PATH:<PetaLinuxDir>/tools/xsct/bin/
```

Also dtc needs to be available therefore either install it directly via your favorite or linux distribution based package manager (e.g. yum, dnf, apt-get).
on Fedora based system:

```
$ sudo yum install dtc
$ sudo dnf install dtc
```

on Debian based system:

```
$ sudo apt-get install device-tree-compiler
```

As the Vivado installation also contains the device tree compiler (dtc), it is also sufficient to source the Vivado settings as an alternative to installing dtc.

```
$ source /<Vivado Installation Dir>/settings64.sh
```

The program emp-xsa-to-overlays.py can now be executed. The .xsa file must be passed as an argument. Execute it as follows:

```
./emp-xsa-to-overlays.py <MyHardwareExport>.xsa
```

All warnings during the process can be ignored! Towards the end, 3 user inputs are required for 3 questions:

- When asked if you wish to "fix pl.dtsi to match EMP requirements?" answer "y".
- When asked if you wish to "make device tree overlay binary?" answer "y".
- When asked if you wish to "clean up your output directory?" answer "y".

Then access the newly created "output" directory and copy the .dtbo and .bit.bin file to the /lib/firmware directory in the root filesystem of the Zynq.

Loading the firmware

The firmware can now be loaded during runtime from the Linux running on PS and performs a device tree integration of the emp_lpgbt devices recognized as uio devices as well as writing the bitstream directly into the FPGA. The shell script ./loadFirmware can be used for this purpose. Execute it as follows:

```
$ sudo epos/etools/loadFirmare.sh <Firmware>.dtbo
```

The tool can also provide you with a list of available .dtbo files. Simply use the -l/--list option as follows:

```
$ sudo epos/etools/loadFirmare.sh -l
```